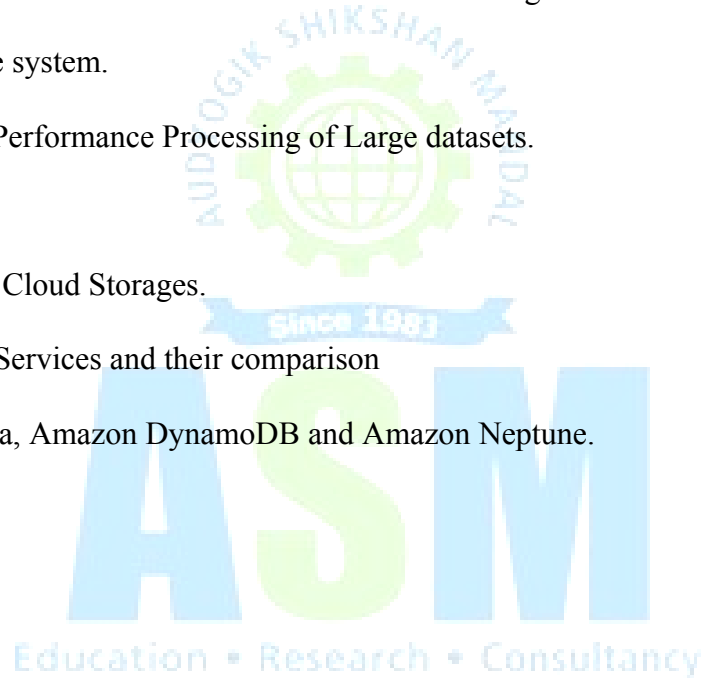
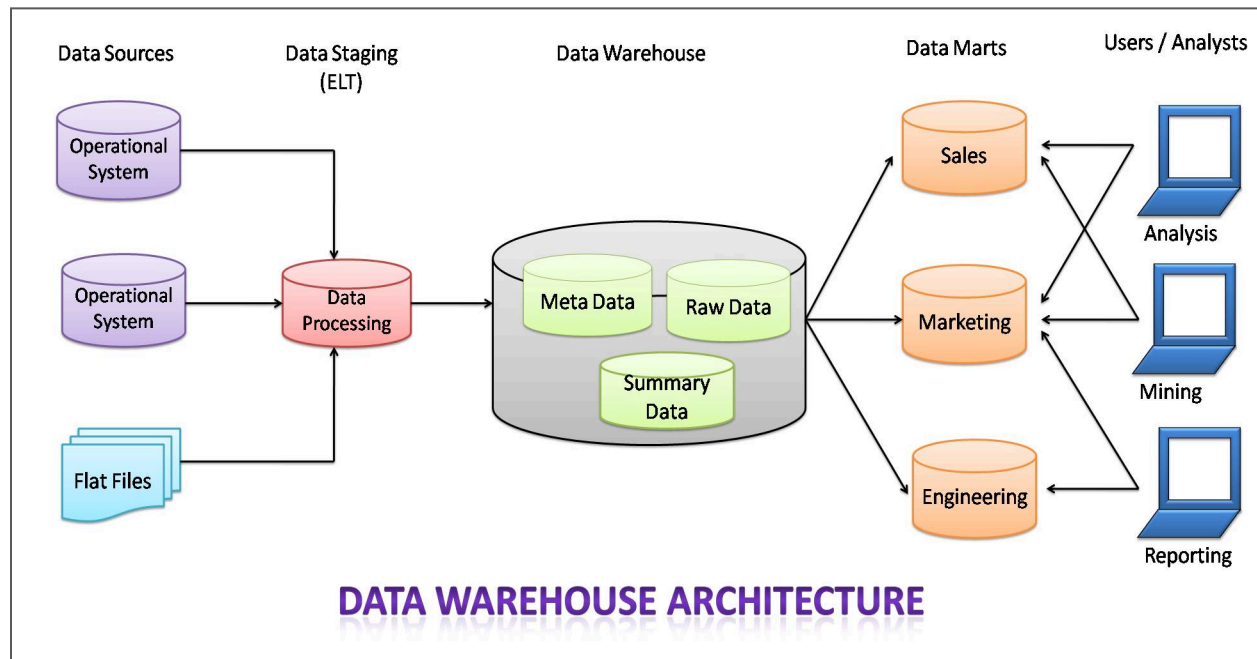


Chapter 2. Fundamentals of Cloud Database and File System

- 2.1 Core concepts of data warehousing.
- 2.2 Primary components and architectures of data warehousing.
- 2.3 Cloud Native file system.
- 2.4 Model for High Performance Processing of Large datasets.
- 2.5 Storage types.
- 2.6 General Purpose Cloud Storages.
- 2.7 Cloud Database Services and their comparison
 - 2.7.1 Amazon Aurora, Amazon DynamoDB and Amazon Neptune.



2.1 Core concepts of data warehousing. (for Exam Draw this diagram)



A **Data Warehouse** is a centralized repository where data from multiple sources is stored, transformed, and organized to support business intelligence (BI) and decision-making processes.

Characteristics of a Data Warehouse

- **Subject-Oriented:** Organized around key subjects (sales, products, customers) rather than applications.
- **Integrated:** Combines data from different sources into a consistent format.
- **Non-Volatile:** Data is stable and doesn't change once entered — historical data is maintained.
- **Time-Variant:** Data is stored with time-related metadata, enabling trend analysis.

ETL Process:

- **Extract:** Pull data from various sources (databases, files, APIs).
- **Transform:** Clean, format, and structure the data.
- **Load:** Store the processed data into the warehouse.

Data Marts:

A **Data Mart** is a subset of a data warehouse, focused on a specific business area (like sales or finance) to improve query performance and user accessibility

OLAP (Online Analytical Processing)

OLAP stands for **Online Analytical Processing**.

It's a category of software tools that allows users to interactively analyze multidimensional data from multiple sources, enabling complex calculations, trend analysis, and data modeling.

A method used to perform multi-dimensional analysis of data. Types of OLAP:

- **ROLAP:** Relational OLAP — data stored in relational databases.
- **MOLAP:** Multidimensional OLAP — data stored in cubes.
- **HOLAP:** Hybrid OLAP — combines ROLAP and MOLAP.

Metadata:

Metadata is data about data — it provides information about the source, structure, and meaning of data within the warehouse. Types:

- **Technical Metadata:** Describes schema, tables, and columns.
- **Business Metadata:** Explains data in business terms.

Data Warehouse Schemas:

- **Star Schema:** A central fact table connected to dimension tables (simple, fast for querying).
- **Snowflake Schema:** Dimension tables are normalized, resulting in a more complex structure.
- **Galaxy Schema:** A combination of star schemas — suitable for complex data models.

2.2 Primary components and architectures of data warehousing.

1. Data Sources:

Data sources are systems that provide raw data for analysis. This data can come from operational databases, flat files, cloud storage, or external APIs. Examples include CRM systems, ERP software, and social media platforms. These systems collect daily business transaction data that is later used for analysis.

2. ETL (Extract, Transform, Load):

ETL is the process of collecting and preparing data for the data warehouse. Extract means gathering data from different sources. Transform means cleaning, formatting, and removing errors or duplicates. Load means storing the processed data into the data warehouse for future

use.

3. Staging Area:

A staging area is a temporary storage space where raw data is kept before entering the data warehouse. In this area, data is cleaned and checked for errors. It ensures that the data is accurate and consistent. After processing, the data is moved to the warehouse.

4. Data Warehouse Storage:

The data warehouse is the main storage system where integrated and historical data is stored. The data is organized using schemas like Star or Snowflake. It helps in analyzing large amounts of information. It can be stored on-premises or in cloud platforms.

5. Data Marts:

Data marts are smaller sections of a data warehouse created for specific departments such as sales or marketing. They contain only relevant data needed by that department. This makes data access faster and easier for users. It helps teams make better decisions quickly.

6. Metadata:

Metadata means data about data. It describes the structure, source, and meaning of the data stored in the warehouse. Technical metadata includes table names and schema details, while business metadata includes KPI definitions and business rules. It helps users understand the data clearly.

7. OLAP (Online Analytical Processing):

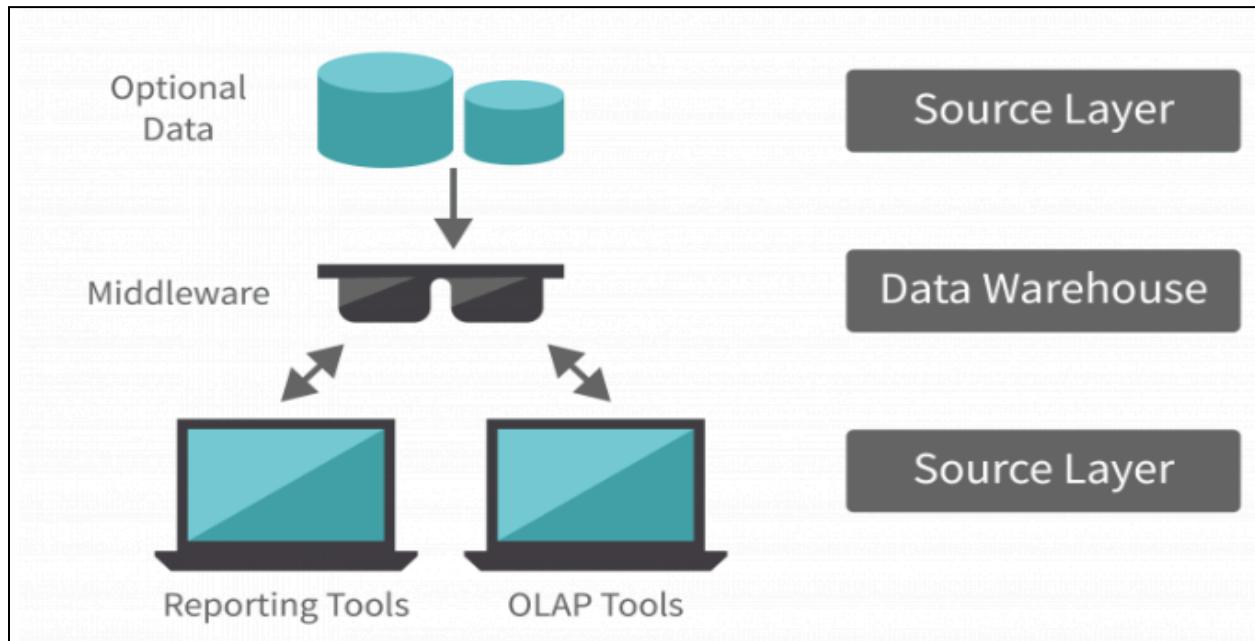
OLAP tools are used to analyze data from different perspectives. They allow users to perform operations like slicing, dicing, drill-down, and roll-up. This makes it easier to study trends and patterns. There are different types of OLAP models such as ROLAP, MOLAP, and HOLAP.

8. Reporting and Visualization:

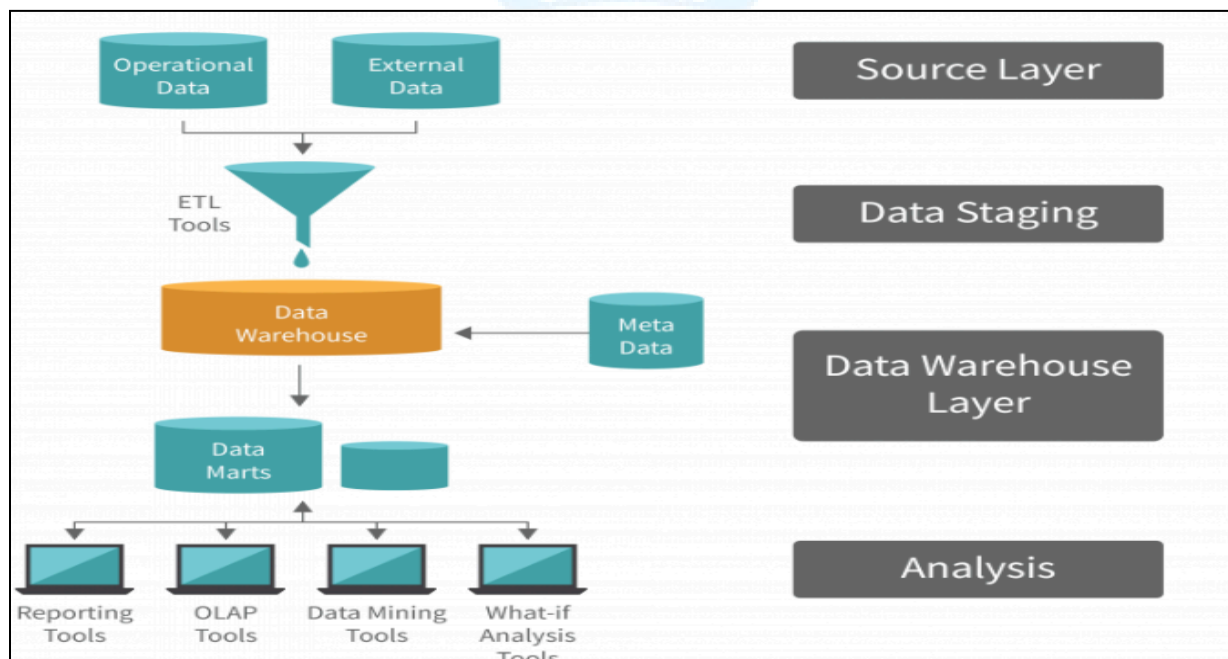
Reporting and visualization tools help users understand data through dashboards, charts, and reports. These tools present complex data in a simple and clear format. They support better decision-making by showing insights visually. Examples include Power BI and Tableau.

Architecture

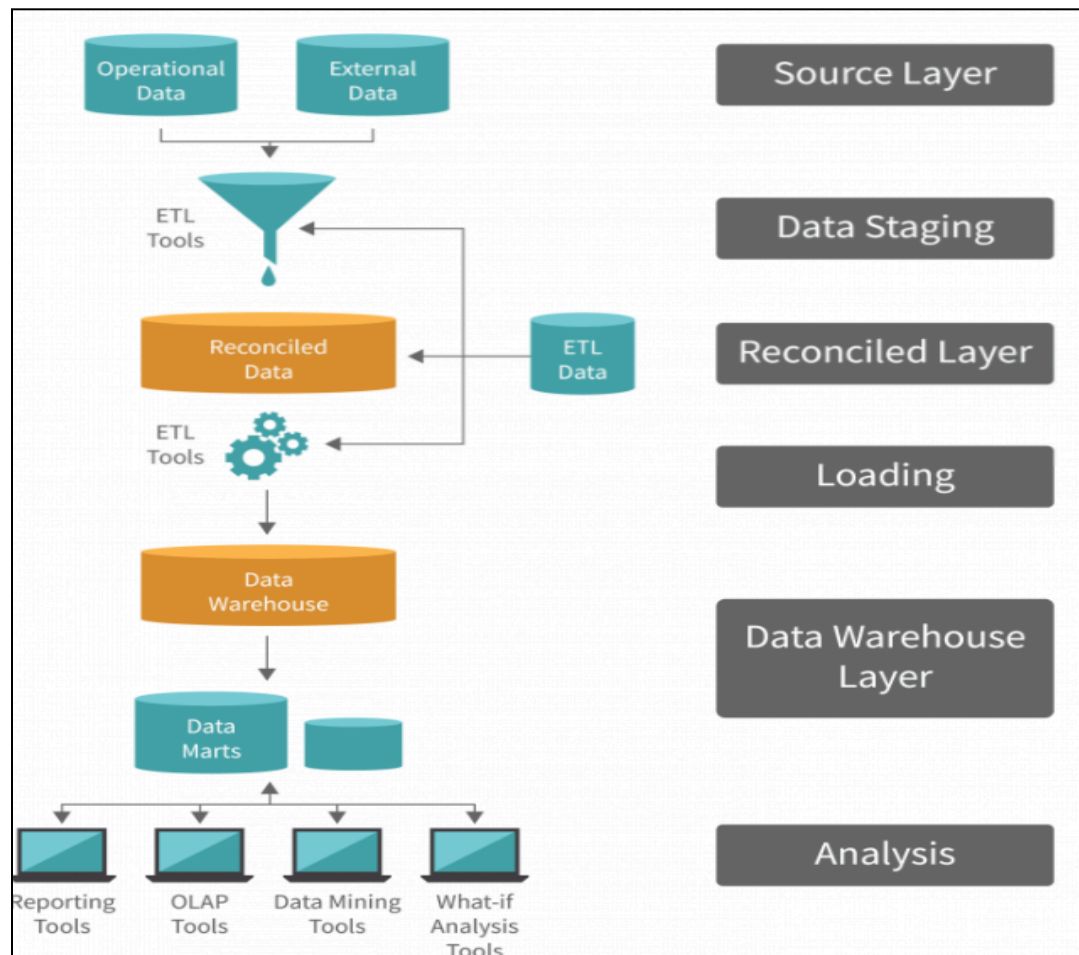
Single-tier Architecture:



Two-tier Architecture:



Three-tier Architecture:



2.3 Cloud Native File System.

A Cloud-Native File System is a modern way of storing and managing files in the cloud instead of on local computers or physical hard drives. The data is stored on remote servers and accessed through the internet using cloud service providers like Amazon Web Services, Google Cloud, and Microsoft Azure. This allows users to access their data anytime and from anywhere, making it convenient and flexible.

The term “**cloud-native**” means the file system is specially designed to fully use cloud features such as scalability, elasticity, automation, and distributed storage. It can automatically increase or decrease storage based on demand and ensures high availability by storing data across multiple servers. Cloud-native file systems also provide strong security, easy integration with applications through APIs, and support for modern technologies like containers and microservices.

Characteristics of a Cloud Native File System

- 1. Scalability:** A cloud native file system can easily scale up or scale down based on user demand. It supports storing massive amounts of data without affecting performance. This makes it suitable for growing businesses and big data applications.
- 2. High Availability :** It ensures that data is available at all times with minimal downtime. Data is replicated across multiple servers or locations. Even if one server fails, the system continues to operate smoothly.
- 3. Distributed Architecture:** Data is stored across multiple machines instead of a single server. This improves performance and balances the workload efficiently. It also increases reliability and reduces the risk of system failure.
- 4. Elasticity:** Resources such as storage and processing power automatically adjust according to workload needs. During high demand, resources increase, and during low demand, they decrease. This helps in cost optimization and efficient resource use.
- 5. Security:** Cloud native file systems provide strong security features like encryption and access control. Identity management ensures only authorized users can access data. Security measures protect sensitive information from threats.
- 6. Performance Optimization:** The system is optimized for high-speed data processing and low latency. It supports parallel data access for better efficiency. This ensures smooth performance for real-time and large-scale applications.
- 7. Accessible:** A cloud native file system is accessible from anywhere through the internet. Users and applications can access data anytime and from any location. It supports multiple devices and platforms, making collaboration easy. Proper authentication and permissions ensure secure and controlled access.

Examples:

- **Amazon EFS** – A file storage service by Amazon Web Services that automatically increases

or decreases storage as needed. It is used for cloud applications.

- **Google Cloud Filestore** – A file storage service provided by Google Cloud that helps store and manage files in the cloud easily.
- **Azure Files** – A cloud file storage service from Microsoft Azure that allows users to store and share files online.
- **CephFS** – An open-source cloud file system used to store large amounts of data safely across multiple servers.
- **GlusterFS** – A distributed file system that stores data across many machines and is used in cloud environments.

Why use a Cloud-Native File System?

- **Always available:** Your files are safe, even if something goes wrong with one server.
- **Pay for what you use:** No need to buy extra storage — you pay only for the space you use.
- **Easy sharing:** Teams can work on files together from different locations.

2.4 Model for High Performance Processing of Large datasets.

When we work with **huge amounts of data** — like analyzing millions of website clicks or processing data from sensors — a normal computer can't handle it alone. That's why we use **models for high-performance data processing**. These models break down the work into smaller pieces and process them at the same time using many computers

Why do we need these models?

- **Too much data:** One computer can't process it all.
- **Speed:** We want results quickly, so we process data in parallel.
- **Scalability:** If data grows, we add more computers instead of upgrading a single one

1. MapReduce Model:

The **MapReduce model** is a programming model used to process very large datasets in a distributed environment. It works in **two main phases: Map and Reduce**.

Map Phase

- The large dataset is divided into smaller parts.
- Each server processes one part of the data.
- It converts the data into **key–value pairs**.

Reduce Phase

- The results from the map phase are collected.
- Similar data is combined and summarized to produce the final result.

Example

Suppose a company wants to count how many times each word appears in **10,000 documents**.

- **Map step:** Each server counts words in a few documents.
- **Reduce step:** The system combines all results and gives the total count of each word.

This model is widely used in the big data framework **Apache Hadoop**.

Advantages

- Processes very large data efficiently
- Uses multiple servers
- Faster processing

2. Distributed Computing Model

In the **distributed computing model**, a large task is divided into many smaller tasks and distributed across multiple computers in the cloud. Each computer works independently but contributes to the final result.

Example: An **online shopping website** stores millions of customer records. Instead of processing all data on one computer, the cloud divides the work across many servers.

For example:

- Server 1 processes customer data from Asia
- Server 2 processes customer data from Europe
- Server 3 processes customer data from America

All results are combined to produce the final analysis. Cloud platforms like **Amazon Web Services** and **Microsoft Azure** use this model.

Advantages

- Faster data processing
- Better scalability
- High reliability

3. Parallel Processing Model

Parallel processing means **many processors work at the same time** to complete a task. Instead of completing tasks one after another (sequential processing), they run **simultaneously**.

Example

A **weather forecasting system** analyzes huge climate datasets.

- Data is divided into multiple parts.
- Each processor analyzes one part at the same time.
- The results are combined to predict weather conditions.

Advantages

- Reduces processing time
- Improves system performance
- Useful for scientific and research applications

4. Data Partitioning Model

In this model, a large dataset is divided into smaller **partitions (chunks)**. Each partition is stored and processed separately on different servers. This approach improves system performance by enabling parallel processing, where multiple servers handle different partitions simultaneously. It also enhances scalability, as new partitions can be added easily when data grows. Additionally, data partitioning helps in efficient resource utilization and reduces the load on a single server, thereby increasing overall system reliability.

Example

A **bank database** contains millions of customer transactions.

The system may divide the data like this:

- Partition 1: Transactions from 2022
- Partition 2: Transactions from 2023
- Partition 3: Transactions from 2024

Each partition can be processed independently, which speeds up analysis.

Advantages

- Easy data management
- Faster query processing
- Efficient storage usage

5. In-Memory Processing Model

In this model, data is processed directly in RAM (main memory) instead of reading it from disk storage. Since RAM is much faster than disk storage, data processing becomes very quick. A popular framework that uses this model is Apache Spark. This approach significantly reduces latency and improves the speed of data analytics, especially for real-time and iterative tasks. It is widely used in big data applications where fast computation and quick insights are required.

Example

A **real-time stock market analysis system** needs to process stock prices instantly.

Instead of saving data on disk first, the system processes it directly in memory, allowing very fast analysis.

Advantages

- Very fast data processing
- Suitable for real-time analytics
- Improves system performance

2.5 Storage types.

1. Object Storage

What it is:

- Stores data as **objects** — each object contains the data, metadata (information about the data), and a unique identifier.
- There are **no folders or directories** — everything is stored in a flat structure (like a giant bucket).
- Best for **unstructured data** — images, videos, backups, and logs.

Key features:

- **Scalable:** Can store unlimited data.
- **Accessible via APIs:** Used for web apps or data lakes.
- **Redundant:** Automatically replicates data across regions for backup.

Examples:

- **Amazon S3 (Simple Storage Service)**
- **Google Cloud Storage**
- **Azure Blob Storage**

A social media app uses **Object Storage** to store millions of profile pictures and videos uploaded by users.

2. File Storage (Network File Storage — NFS)

What it is:

- Organizes data in **folders and directories** — similar to how we store files on a computer (C:/Documents/).
- Great for **shared access** — multiple users or applications can access the same file at the same time.

Key features:

- **Structured storage:** Supports file systems like NFS (Network File System) and SMB (Server Message Block).
- **Collaborative:** Ideal for applications where many users need to access shared files.
- **Scalable:** Can grow to petabytes of data.

Examples:

- **Amazon EFS (Elastic File System)**
- **Google Filestore**
- **Azure Files**

Real-life example:

A software development team stores their project files on cloud **File Storage**, so all developers can access, edit, and test code together.

3. Block Storage

What it is:

- Splits data into **fixed-sized blocks** — like how your computer's hard drive works.
- Each block has a unique address, making it fast to read/write data.
- Used for **structured data** — databases, operating systems, and virtual machines (VMs).

Key features:

- **Ultra-fast:** Low latency and high performance — perfect for apps that need quick access to data.
- **Attachable to VMs:** Works like a cloud-based hard drive.
- **Customizable:** You can resize or snapshot storage volumes.

Examples:

- **Amazon EBS (Elastic Block Store)**
- **Google Persistent Disk**
- **Azure Disk Storage**

Real-life example:

An online shopping website uses **Block Storage** to store customer databases and support high-speed transactions.

4. Cold Storage (Archive Storage)

What it is:

- Stores data you don't use often — like backups, archives, and legal records.
- Prioritizes **cost savings** over fast access.
- Accessing data can take minutes or hours.

Key features:

- **Extremely cheap:** Pay less to store data long-term.
- **Retrieval delay:** Takes time to "unfreeze" data before you can access it.
- **Versioning and encryption:** Keeps old versions of files for data recovery.

Examples:

- **Amazon Glacier**
- **Google Archive Storage**
- **Azure Blob Archive Tier**

Real-life example:

A hospital uses **Cold Storage** to keep patient records for 10 years, only accessing them when needed for legal or medical review.

How to choose the right cloud storage:

1. Need fast access to databases or VMs? → Block Storage
2. Collaborating on team projects or shared files? → File Storage
3. Storing millions of images, videos, or backups? → Object Storage
4. Long-term storage for backups or archives? → Cold Storage

Real-world use cases:

- **Netflix:** Uses **Object Storage (AWS S3)** to store video content and stream it to users.
- **Dropbox:** Uses **File Storage** to allow users to store and share files.
- **Amazon:** Uses **Block Storage (EBS)** to support fast e-commerce transactions.
- **NASA:** Uses **Cold Storage** to archive years of satellite data.

2.6 General Purpose Cloud Storages.

1. Amazon S3 (Simple Storage Service):

- **Type:** Object storage
- **Best for:** Storing images, videos, backups, and logs.
- **Features:**
 - ➔ Highly scalable — can store unlimited data.
 - ➔ Supports versioning (keeps old file versions).

- Integrates with AI and analytics tools.
- **Use case:** Hosting static websites or storing backups.

2. Google Cloud Storage:

- **Type:** Object storage
- **Best for:** Web content, big data analytics, or disaster recovery.
- **Features:**
 - ➔ Different storage classes — Standard, Nearline, Coldline, and Archive.
 - ➔ Automatic data lifecycle management.
- **Use case:** Analyzing large datasets for business insights.

3. Microsoft Azure Blob Storage:

- **Type:** Object storage
- **Best for:** Unstructured data like documents, images, and logs.
- **Features:**
 - ➔ Stores massive amounts of data.
 - ➔ Integrates easily with Microsoft apps (Office 365, etc.).
- **Use case:** Storing media files for a streaming service.

4. Amazon EBS (Elastic Block Store):

- **Type:** Block storage
- **Best for:** Attaching storage volumes to cloud-based virtual machines (EC2 instances).
- **Features:**
 - ➔ Low latency — super fast data access.
 - ➔ Snapshots for backups.
- **Use case:** Running cloud databases or apps that need fast storage.

5. Google Persistent Disk:

- **Type:** Block storage
- **Best for:** Running cloud VMs and databases.
- **Features:**
 - Data encryption by default.
 - Easily scales up or down.
- **Use case:** Supporting cloud applications that need constant data access.

6. Azure Files:

- **Type:** File storage
- **Best for:** Cloud-based shared drives.

- **Features:**
 - SMB protocol support — like a cloud-based Windows file server.
 - Access data from anywhere.
- **Use case:** Storing documents and files for a remote working team.

2.7 Cloud Database Services and their comparison

What is a Cloud Database?

Imagine you have a huge project that needs a lot of data storage. Instead of buying a hard drive, setting up a powerful computer, and installing database software (which is costly and time-consuming), you simply rent it over the internet.

A Cloud Database is exactly that. It is a database (like MySQL, PostgreSQL, MongoDB, etc.) that runs on a cloud computing platform (like Amazon Web Services, Google Cloud, or Microsoft Azure). You don't own the hardware; you just use it and pay for what you use.

The Cloud Solution:

A Cloud Database is a database that is hosted, managed, and provided to you over the internet (the cloud).

- You don't own the hardware. You rent it.
- You don't install the software. It's ready to use.
- You don't manage the backups. The cloud provider does it for you.

Types of Cloud Databases

A) Relational Databases (SQL Databases)

- Data is stored in **tables (rows and columns)**
- Uses **Structured Query Language (SQL)**
- Suitable for structured data

Examples: MySQL, PostgreSQL

Cloud Services:

- Amazon RDS
- Azure SQL Database
- Google Cloud SQL

B) Non-Relational Databases (NoSQL Databases)

- Stores **unstructured or semi-structured data**
- No fixed table format
- Flexible and scalable

Types of NoSQL:

- Document-based (MongoDB)
- Key-Value (DynamoDB)
- Graph-based

Cloud Services:

- Amazon DynamoDB
- Google Firestore

Relational Database (SQL)	Non-Relational Database (NoSQL)
It stores data in tables with rows and columns (like a spreadsheet).	It stores data in documents, key-value pairs, graphs, or wide columns (like a JSON file).
It has a fixed schema, meaning you must define the structure (columns) before adding data.	It has a dynamic schema, meaning each record can have different fields without prior definition.
It is designed to scale vertically, which means upgrading to a bigger, more powerful server.	It is designed to scale horizontally, which means adding more cheap servers to distribute the load.
It uses SQL (Structured Query Language) to interact with data.	It uses APIs or Unstructured Query Languages (like MongoDB Query Language).
It follows ACID properties (Atomicity, Consistency, Isolation, Durability) to ensure reliable transactions.	It follows BASE properties (Basically Available, Soft state, Eventual consistency) for flexibility and performance.
It is best for complex queries, joins, and	It is best for large-scale, high-velocity data with

multi-row transactions.	simple queries.
It is ideal for applications where data integrity and consistency are critical (e.g., Banking, ERP).	It is ideal for applications where speed and scalability are critical (e.g., Social media, Gaming, IoT).
Examples: MySQL, PostgreSQL, Oracle, SQL Server.	Examples: MongoDB, DynamoDB, Cassandra, Redis, Firebase.
Cloud examples: Amazon RDS, Azure SQL Database, Cloud SQL.	Cloud examples: Amazon DynamoDB, Azure Cosmos DB, Firestore.
It handles relationships between data very well using Foreign Keys and Joins.	It handles relationships poorly; data is often denormalized or nested within the same record.

Popular Cloud Database Services

1. Amazon RDS (Relational Database Service)

- Provided by Amazon Web Services, Amazon RDS is a managed **SQL database service** that supports MySQL, PostgreSQL, and Oracle.
- It handles backups, updates, and maintenance automatically. It is best for structured data applications.
- It works like a ready-to-use database where you don't need to install or manage anything. You just store and access your data easily.
- **Example:** College student database system

2. Amazon DynamoDB

- Also provided by Amazon Web Services, DynamoDB is a **NoSQL database** designed for high speed and scalability.
- It stores data in key-value format and is used for real-time applications.
- It is very fast and can handle a large amount of data without slowing down. It is useful when data changes quickly.

- **Example:** Online shopping apps, gaming apps

3. Azure SQL Database

- Offered by Microsoft Azure, this is a fully managed **relational database service** with high security and performance.
- It integrates well with Microsoft tools.
- It is easy to use for organizations already using Microsoft software like Windows or .NET.
- It keeps data secure and well-organized.
- **Example:** Banking systems, enterprise applications

4. Google Cloud SQL

- Provided by Google Cloud Platform, it is a managed SQL database supporting MySQL and PostgreSQL.
- It is easy to use and suitable for web applications.
- It helps developers quickly create and manage databases without technical complexity.
- It is good for small to medium applications.
- **Example:** Website backend database

5. Google Firestore

- Firestore is a **NoSQL database** from Google Cloud Platform that provides real-time data synchronization.
- It is widely used in mobile and web applications.
- It updates data instantly across all users, which is useful for apps where live updates are needed.
- **Example:** Chat applications, real-time apps

2.7.1 Amazon Aurora, Amazon DynamoDB and Amazon Neptune.

Amazon offers different types of cloud databases to handle different types of data and workloads

1. Amazon Aurora (Relational Database - SQL)

- A fast and reliable **relational database** service.
- Compatible with **MySQL and PostgreSQL** (popular database systems).
- Stores data in structured tables (rows & columns).

Why use it?

- **Faster than traditional databases** (5x MySQL, 3x PostgreSQL).
- **Automatic Scaling** – Grows storage up to **128 TB** as needed.
- **Highly available** (data is backed up across multiple locations).
- **Security** – Supports **encryption, backups, and access control** to protect data.

Best for:

- Banking & financial applications.
- Business software (ERP, CRM).
- E-commerce websites.

Amazon DynamoDB (NoSQL Database - Key-Value & Document Store)

- A **NoSQL database** designed for high-speed performance.
- Stores data as **key-value pairs** or **documents** instead of tables.

Why use it?

- **Handles millions of requests per second** (high performance).
- **Auto Scaling** – Can handle **millions of requests per second** without downtime.
- **Fully Managed & Serverless** – No need to manage servers; AWS handles everything.
- **Security** – Supports encryption, access control, and compliance with industry standards.

Best for:

- Mobile & gaming applications.
- IoT (Internet of Things) data storage.
- Real-time apps like chat and shopping carts.

3. Amazon Neptune (Graph Database - Relationships & Networks)

- A **graph database** that stores data in a connected way.
- Used to find relationships between different data points.

Why use it?

- **Super fast for relationship-based queries.**
- **Supports popular graph models** (Property Graph & RDF).
- **Best for complex network data** (social media, fraud detection).

Best for:

- Social networks
- Fraud detection in banking.
- Recommendation systems (e.g., Netflix, Amazon).

Which One to Choose?

- Need structured data with SQL? → **Use Amazon Aurora.**
- Handling fast-growing, flexible data? → **Go for DynamoDB.**
- Working with relationships & networks? → **Choose Amazon Neptune.**

Feature	Amazon Aurora	Amazon DynamoDB	Amazon Neptune
Database Type	Relational (SQL)	NoSQL	Graph Database
Data Model	Tables (rows & columns)	Key-Value / Document	Nodes & Edges
Schema	Fixed schema	Flexible schema	Flexible schema
Query Language	SQL	API / PartiQL	Gremlin / SPARQL
Scalability	Auto-scaling (limited horizontal)	Fully serverless, highly scalable	Scales for relationship queries
Performance	High (optimized transactions)	Very high (low latency)	High for connected data
Best Use Case	Banking, ERP, transactional apps	Gaming, IoT, real-time apps	Social networks, recommendations
Data Relationships	Strong relationships (join)	Limited relationships	Complex relationships

Storage Type	Structured data	Semi-structured / unstructured	Highly connected data
Management	Managed service	Fully managed (serverless)	Fully managed
Backup & Recovery	Automatic backups	Built-in backup	Automatic backups
Cost Model	Pay for instances & storage	Pay-per-request	Pay for instances & storage

